

Data Structures



Submitted to

MR. Bilal Bukhari

Submitted by

Laraib Khalid

(fa25-bcs-059)

COMSATS University Islamabad,

Attock Campus.

Assignment No 1

CODE:

```
#include <iostream>

using namespace std;

const int CAPACITY = 20;

// ArrayList structure

struct ArrayList
{
    int data[CAPACITY];
    int size = 0;
};

// Insert value at end

bool insertEnd(ArrayList &list, int value)
{
    if (list.size >= CAPACITY)
        return false;

    list.data[list.size] = value;
    list.size++;
    return true;
}

// Insert value at beginning

bool insertAtBeginning(ArrayList &list, int value)
{
    if (list.size >= CAPACITY)
        return false;
```

```
    for (int i = list.size; i > 0; i--)
    {
        list.data[i] = list.data[i - 1];
    }
    list.data[0] = value;
    list.size++;
    return true;
}
```

// Delete value from a position

```
bool deleteAtPosition(ArrayList &list, int position)
{
    if (position < 0 || position >= list.size)
        return false;
    for (int i = position; i < list.size - 1; i++)
    {
        list.data[i] = list.data[i + 1];
    }
    list.size--;
    return true;
}
```

// Display ArrayList

```
void displayList(const ArrayList &list)
{
    for (int i = 0; i < list.size; i++)
    {
        cout << list.data[i] << " ";
    }
}
```

```

    }

    cout << endl;
}

int main()
{
    ArrayList list;

    // Change 8 to the last two digits of your registration number

    int regNo = 8;

    // Pointers

    int *ptr = list.data;

    int *minPtr = list.data;

    int *maxPtr = list.data;

    int *medianPtr = list.data;

    int *closestPtr = list.data;

    int sum = 0;

    int closestPosition;

    double generalAverage;

    double specialAverage;

    double averageDifference;

    double finalScore;

    // =====

    // PART A - Insert 9 values

    // =====

    insertEnd(list, 18);

    insertEnd(list, 7);

```

```
insertEnd(list, 45);
insertEnd(list, 11);
insertEnd(list, 36);
insertEnd(list, 59);
insertEnd(list, 21);
insertEnd(list, 13);
insertEnd(list, 29);
cout << "Initial ArrayList: ";
displayList(list);
```

```
// =====
```

```
// PART B - Pointer Traversal, Sum, Min and Max
```

```
// =====
```

```
ptr = list.data;
minPtr = list.data;
maxPtr = list.data;
sum = 0;
for (int i = 0; i < list.size; i++)
{
    sum = sum + *ptr;
    if (*ptr < *minPtr)
        minPtr = ptr;
    if (*ptr > *maxPtr)
        maxPtr = ptr;
    ptr++;
}
```

```

}

cout << "Minimum Value: " << *minPtr << endl;
cout << "Maximum Value: " << *maxPtr << endl;
cout << "Sum: " << sum << endl;


// =====

// PART C - Find Median

// =====


int temp[CAPACITY];
// Copy original ArrayList into temporary array
for (int i = 0; i < list.size; i++)
{
    temp[i] = list.data[i];
}

// Bubble Sort
for (int i = 0; i < list.size - 1; i++)
{
    for (int j = 0; j < list.size - i - 1; j++)
    {
        if (temp[j] > temp[j + 1])
        {
            int x = temp[j];
            temp[j] = temp[j + 1];
            temp[j + 1] = x;
        }
    }
}

```

```

    }
}

// 9 values -> middle index is 4
int medianValue = temp[list.size / 2];

// Find median in original ArrayList
medianPtr = list.data;
for (int i = 0; i < list.size; i++)
{
    if (*medianPtr == medianValue)
        break;
    medianPtr++;
}
cout << "Median Value: " << *medianPtr << endl;

// =====
// PART D - Averages and Closest Value
// =====

generalAverage = (double)sum / list.size;
specialAverage =
    (*minPtr + *medianPtr + *maxPtr) / 3.0;
cout << "General Average: " << generalAverage << endl;
cout << "Special Average: " << specialAverage << endl;

// Find closest value
ptr = list.data;

```

```

closestPtr = list.data;

double smallestDifference = 1000000;

for (int i = 0; i < list.size; i++)
{
    double difference = *ptr - specialAverage;

    if (difference < 0)
        difference = -difference;

    if (difference < smallestDifference)
    {
        smallestDifference = difference;

        closestPtr = ptr;
    }

    ptr++;
}

closestPosition = closestPtr - list.data;

cout << "Closest Value: " << *closestPtr << endl;

cout << "Position of Closest Value: "

    << closestPosition << endl;

// =====

// PART E - Final Calculations

// =====

averageDifference =

    generalAverage - specialAverage;

```

```

if (averageDifference < 0)
    averageDifference = -averageDifference;
finalScore =
    (*closestPtr - generalAverage < 0 ?
    -( *closestPtr - generalAverage ) :
    ( *closestPtr - generalAverage ))
    +
    (*closestPtr - specialAverage < 0 ?
    -( *closestPtr - specialAverage ) :
    ( *closestPtr - specialAverage ))
    +
    averageDifference;
cout << "Difference Between Averages: "
    << averageDifference << endl;
cout << "Final Score: " << finalScore << endl;

```

```
// =====
```

```
// Delete closest value
```

```
// =====
```

```

closestPosition = closestPtr - list.data;
deleteAtPosition(list, closestPosition);
cout << "ArrayList After Deletion: ";
displayList(list);

```

```
// =====
```

```

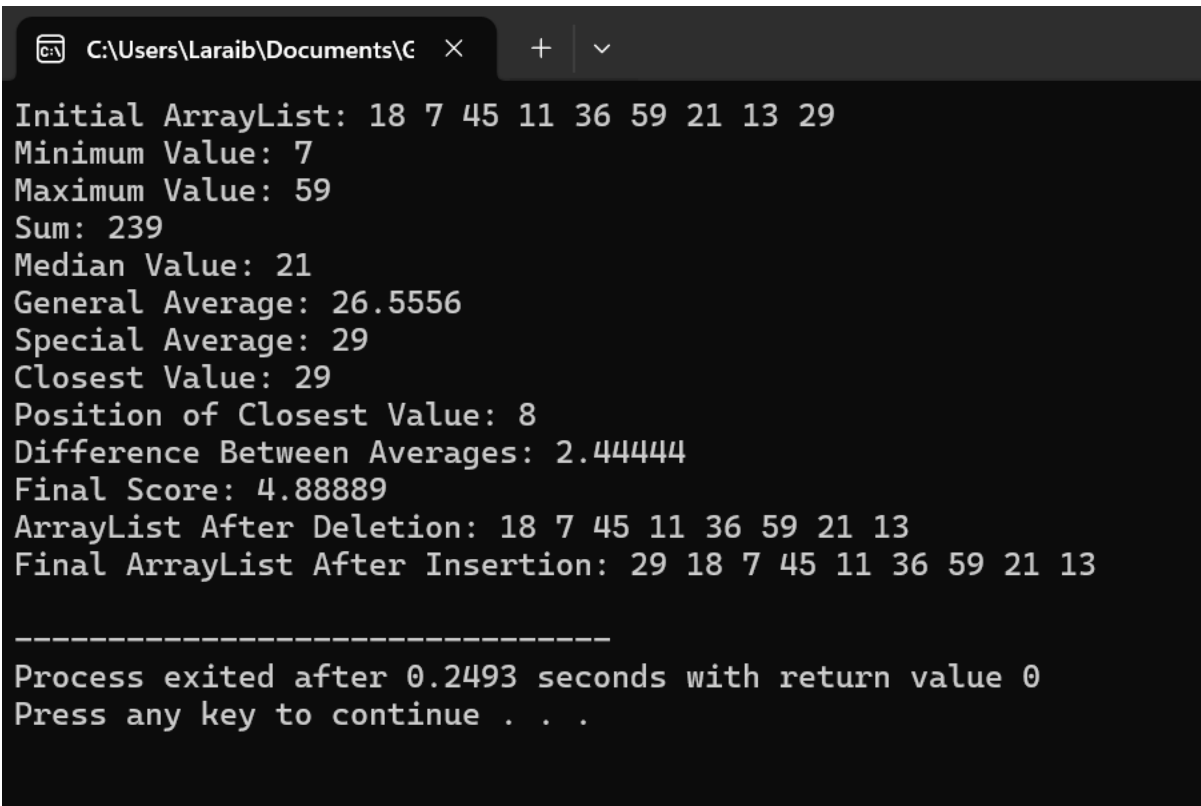
// Round Special Average and Insert at Beginning

// =====

int roundedSpecialAverage =
    (int)(specialAverage + 0.5);
insertAtBeginning(list, roundedSpecialAverage);
cout << "Final ArrayList After Insertion: ";
displayList(list);
return 0;
}

```

RESULT:



```

C:\Users\Laraib\Documents\C x + v
Initial ArrayList: 18 7 45 11 36 59 21 13 29
Minimum Value: 7
Maximum Value: 59
Sum: 239
Median Value: 21
General Average: 26.5556
Special Average: 29
Closest Value: 29
Position of Closest Value: 8
Difference Between Averages: 2.44444
Final Score: 4.88889
ArrayList After Deletion: 18 7 45 11 36 59 21 13
Final ArrayList After Insertion: 29 18 7 45 11 36 59 21 13

-----
Process exited after 0.2493 seconds with return value 0
Press any key to continue . . .

```